

```
In [1]: import os
import sys
sys.path.append(os.path.dirname(os.getcwd()))

from query_helper import get_connection, run_query

conn = get_connection()
```

Connection to database successful.

```
In [2]: """
Detect visits that do not have a corresponding billing record.
"""
_ = run_query(conn, """
SELECT
    visits.visit_id,
    visits.patient_id,
    visits.visit_date
FROM visits
LEFT OUTER JOIN billing ON visits.visit_id = billing.visit_id
WHERE billing.visit_id IS NULL;
""")
```

```
=====
Query
=====
```

 Query Plan:
SCAN visits
SEARCH billing USING COVERING INDEX idx_billing_visit_id (visit_id=?) LEFT-JOIN

 Results (0 rows):

visit_id	patient_id	visit_date
----------	------------	------------

```
In [3]: """
Detect billing records that do not have a corresponding visit.
"""
_ = run_query(conn, """
SELECT
    billing.bill_id,
    billing.visit_id,
    billing.billed_amount
FROM billing
LEFT OUTER JOIN visits ON billing.visit_id = visits.visit_id
WHERE visits.visit_id IS NULL;
""")
```

```
=====
Query
=====
```

 Query Plan:
SCAN billing
SEARCH visits USING INTEGER PRIMARY KEY (rowid=?) LEFT-JOIN

 Results (0 rows):

bill_id	visit_id	billed_amount
---------	----------	---------------

In [4]:

```
"""
Identify patients with duplicate patient_id values
"""

_ = run_query(conn, """
SELECT
    patient_id,
    COUNT(*) AS count
FROM patients
GROUP BY patient_id
HAVING COUNT(*) > 1;
""")
```

```
=====
Query
=====
```

 Query Plan:
SCAN patients

 Results (0 rows):

patient_id	count
------------	-------

In [5]:

```
"""
Find records with missing or invalid length_of_stay_hours or payment_days.
"""

_ = run_query(conn, """
SELECT
    visits.visit_id,
    visits.length_of_stay_hours,
    billing.payment_days,
    billing.claim_status,
    case when visits.length_of_stay_hours IS NULL OR visits.length_of_stay_hours < 0 then 'Invalid length_of_stay_hours'
         when (billing.payment_days IS NULL OR billing.payment_days < 0) AND billing.claim_status = 'Paid' then 'Invalid payment_days'
         else 'Valid' end AS validation_status
FROM visits
    INNER JOIN billing ON visits.visit_id = billing.visit_id
WHERE
    visits.length_of_stay_hours IS NULL
    OR (billing.payment_days IS NULL AND billing.claim_status = 'Paid')
    OR visits.length_of_stay_hours < 0
    OR (billing.payment_days < 0 AND billing.claim_status = 'Paid');
```

```
""""
)
```

Query

 Query Plan:
SCAN billing
SEARCH visits USING INTEGER PRIMARY KEY (rowid=?)

 Results (459 rows):

	visit_id	length_of_stay_hours	payment_days	claim_status	validation_status
0	3	34.36	None	Paid	Invalid payment_days
1	22	15.27	None	Paid	Invalid payment_days
2	41	18.26	None	Paid	Invalid payment_days
3	66	9.59	None	Paid	Invalid payment_days
4	120	44.21	None	Paid	Invalid payment_days
...	...	...	...	...	...
454	24842	10.48	None	Paid	Invalid payment_days
455	24851	27.29	None	Paid	Invalid payment_days
456	24896	18.86	None	Paid	Invalid payment_days
457	24944	26.58	None	Paid	Invalid payment_days
458	24988	13.85	None	Paid	Invalid payment_days

459 rows × 5 columns

```
In [6]: """
Identify visits linked to patients with missing insurance provider information.
"""
_ = run_query(conn, """
SELECT
    visits.visit_id,
    visits.patient_id,
    patients.insurance_provider
FROM visits
INNER JOIN patients ON visits.patient_id = patients.patient_id
WHERE patients.insurance_provider IS NULL;
""")
```

=====

Query

=====

 Query Plan:
SEARCH patients USING COVERING INDEX idx_patients_insurance (insurance_provider=?)
SEARCH visits USING COVERING INDEX idx_visits_patient_id (patient_id=?)

 Results (0 rows):

visit_id	patient_id	insurance_provider
----------	------------	--------------------